

Solution 15 December, 2023

Course title: Network Optimization

Course no: 42115

Aids allowed: All

Exam duration: 4 hours

Critical path with earliest start times

Thristur (pristur) is the favorite mini-candy in Iceland. Production consists of the following six steps:

activity	description	duration	predecessor	earliest starting time
<i>A</i>	Cut toffee into squares	2	<i>F</i>	8
<i>B</i>	Cut licorice into pieces	1		
<i>C</i>	Cover with chocolate	3	<i>A, E</i>	
<i>D</i>	Mix caramel cream	3		
<i>E</i>	Melt chocolate	4	<i>B, F</i>	7
<i>F</i>	Mix caramel cream and licorice	4	<i>D, B</i>	

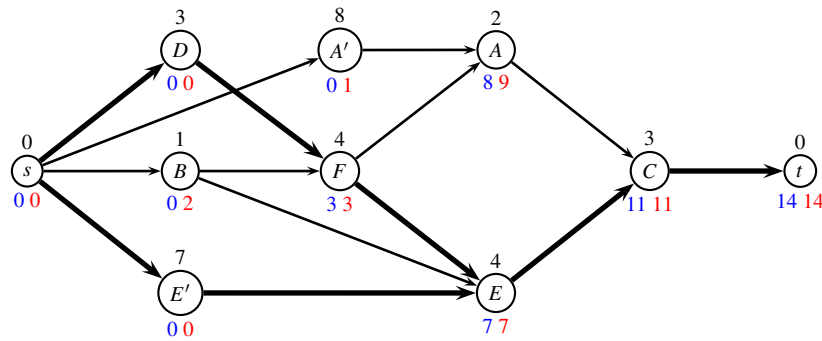
Since some of the ingredients are made in remote factories, we have earliest starting times on some of the activities.

Answer 1 We add a node A' with duration 8 and no predecessors. We include A' as predecessor to A . We add a node E' with duration 7 and no predecessors. We include E' as predecessor to E . In this way we get:

activity	description	duration	predecessor
A'	Earliest time of A	8	
A	Cut toffee into squares	2	F, A'
B	Cut licorice into pieces	1	
C	Cover with chocolate	3	A, E
D	Mix caramel cream	3	
E'	Earliest time of E	7	
E	Melt chocolate	4	B, F, E'
F	Mix caramel cream and licorice	4	D, B

Tasks without a predecessor have start-node s as predecessor. ■

Answer 2 A topological ordering of the activities must ensure that all edges are pointing forward. This is the case in the following graph:



■

Answer 3 Let U_i be the earliest starting time of task i (marked with blue), and let V_i be the latest starting time (marked with red).

- *Forward procedure:* Starting with $U_s = 0$, we use recursion $U_j = \max_{(i,j) \in E} (U_i + d_i)$.
- *Makespan:* $T = U_t = 14$.
- *Backward procedure:* Starting with $V_t = T$, we use recursion $V_i = \min_{(i,j) \in E} (V_j - d_i)$.

Note: If you solved the original problem then earliest/latest times: D 0 0, B 0 2, F 3 3, A 7 9, E 7 7, C 11 11. ■

Answer 4 The critical path is found where $U_i = V_j$ (fat edges on above figure). Notice that we have two critical paths.

Note: If you solved the original problem then the critical path is $sDFECt$. ■

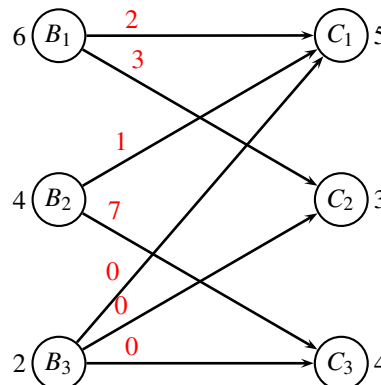
Transportation problem

Two bakeries B_1 and B_2 are delivering cakes to three courses C_1 (42115), C_2 (42114) and C_3 (42142). The bakeries are producing 6 and 4 cakes respectively. Course C_1 can eat at most 5 cakes, course C_2 can eat at most 3 cakes, and course C_3 can eat at most 4 cakes. All cakes produced need to be eaten. The transportation costs are given by the following table

	C_1	C_2	C_3	production
B_1	2	3	-	6
B_2	1	-	7	4
max demand	5	3	4	

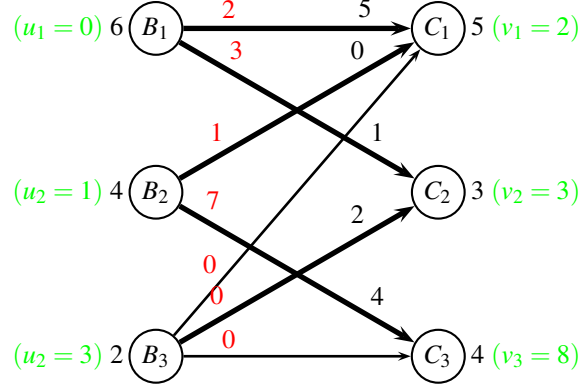
A “-” indicates that transportation is not possible. The objective is to minimize overall transportation costs. **Note,** that the problem is not balanced.

Answer 5 We add a dummy node B_3 to model unmet demand. The dummy node has production $(5 + 3 + 4) - (6 + 4) = 2$. The dummy node is connected to all courses C_j with edges having cost 0.



■

Answer 6 We first consider B_1 , and send 5 units to C_1 , and then 1 units to C_2 . Next, we consider B_2 , and send 4 units to C_3 . Finally, we consider B_3 , and send 2 units to C_2 . This gives the following greedy solution:



The cost of the solution is $z = 2 \cdot 5 + 3 \cdot 1 + 7 \cdot 4 + 0 \cdot 2 = 41$.

Fat edges on the graph indicate the basis. Since we have only 4 basis variables, we add an additional fat edge (B_2, C_1) .

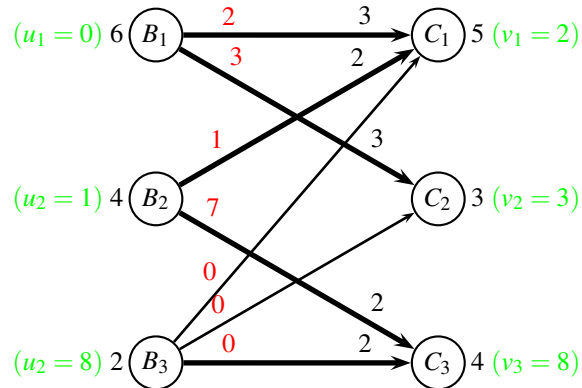
Dual values (indicated with green) are calculated by finding the shortest path from node B_1 , using only the fat edges. ■

Answer 7 We now find the reduced costs as $\bar{c}_{ij} = c_{ij} + u_i - v_j$, getting

$$\begin{aligned}\bar{c}_{31} &= 0 + 3 - 2 = 1 \\ \bar{c}_{33} &= 0 + 3 - 8 = -5\end{aligned}$$

We choose edge (B_3, C_3) corresponding to the cycle $(B_3, C_3, B_2, C_1, B_1, C_2)$. We can send at most $\delta = \min\{4, 5, 2\} = 2$ units along this cycle.

After the pivot operation we get the solution



The cost of the solution is $z = 2 \cdot 3 + 3 \cdot 3 + 1 \cdot 2 + 7 \cdot 2 + 0 \cdot 2 = 31$. Dual variables are calculated as before. The reduced costs are

$$\begin{aligned}\bar{c}_{31} &= 0 + 8 - 2 = 6 \\ \bar{c}_{32} &= 0 + 8 - 3 = 5\end{aligned}$$

No negative reduced costs, so we terminate. The optimal solution value is $z = 31$. ■

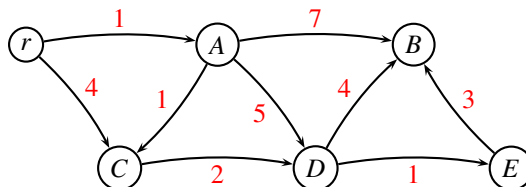
Checking:

```
minimize 2 x11 + 3 x12 + 1 x21 + 7 x23 + 0 x31 + 0 x32 + 0 x33
subject to
  x11 + x12 = 6
  x21 + x23 = 4
  x31 + x32 + x33 = 2
  x11 + x21 + x31 = 5
  x12 + x32 = 3
  x23 + x33 = 4
end
```

```
CPLEX> opt
Dual simplex - Optimal:  Objective =  3.1000000000e+01
CPLEX> dis sol var -
Variable Name      Solution Value
x11                 3.000000
x12                 3.000000
x21                 2.000000
x23                 2.000000
x33                 2.000000
All other variables in the range 1-7 are 0.
```

Shortest Path Problem

Consider the following directed graph where distances $w_{ij} \geq 1$ are indicated with red.



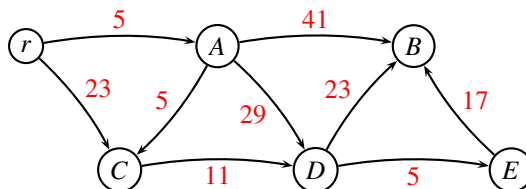
We want to find the shortest path from r to all nodes. If several paths have the same length, we are interested in the shortest path containing *most* edges.

Answer 8 Let $N = 6$ be the number of nodes. This is an upper bound on the longest path. Now, set the edge weights to

$$w'_{ij} = Nw_{ij} - 1$$

This will mean that we still find the shortest path with respect to the original weights w_{ij} but get a reward of 1 for each edge included. Furthermore, notice that since all $w_{ij} \geq 1$ then the new weights w'_{ij} will be non-negative.

This results in the following graph:



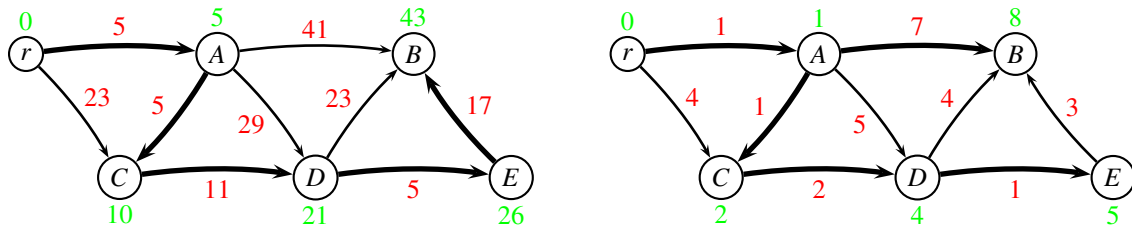
Note: Alternative formulation is: $w'_{ij} = w_{ij} - \epsilon$ where ϵ is a small number, e.g. 0.1. ■

Answer 9 In the following table we report the development of the shortest distances y_i for each node i . The first column indicates which node is considered in Dijkstra's algorithm. (**Note:** right table for original problem)

	r	A	B	C	D	E
init	0	∞	∞	∞	∞	∞
r		5		23		
A			46	10	34	
C					21	
D			44			26
E			43			
B						

	r	A	B	C	D	E
init	0	∞	∞	∞	∞	∞
r		1		4		
A			8	2	6	
C					4	
D						5
E						
B						

The shortest path tree is (**Note:** right figure for original problem)



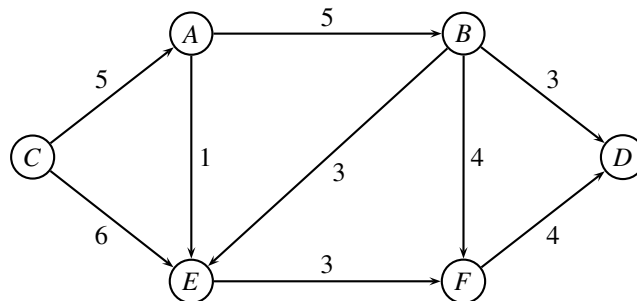
We can next use the shortest path tree to find the original costs and edges:

	r	A	B	C	D	E
shortest path	0	1	8	2	4	5
edges	0	1	5	2	3	4

■

Circulation problem / Maximum flow with demands

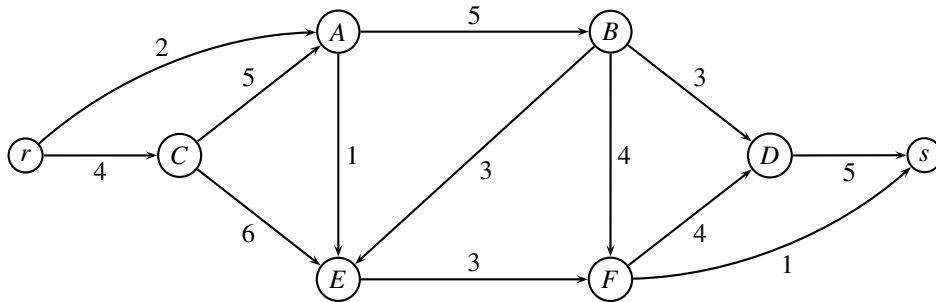
Given a directed graph $G = (V, E)$ and associated edge capacities u_{ij} for each edge (i, j) . In the *Circulation Problem* we have multiple sources and multiple sinks. If a node has positive demand $d_i > 0$ it is called a sink. If a node has negative demand $d_i < 0$ it is called a source. Obviously, we need to have $\sum_{i \in V} d_i = 0$ to match supply and demand.



Assume that we have demands

node i	A	B	C	D	E	F
demand d_i	-2	0	-4	5	0	1

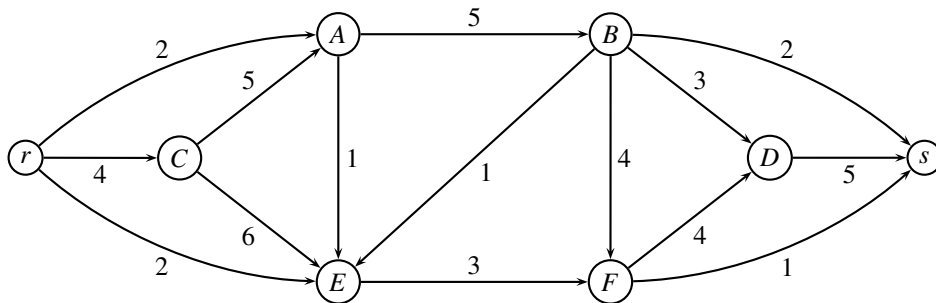
Answer 10 We introduce a root node r and connect it to all nodes having a negative demand. The capacity of the edge is equal to $-d_i$. We introduce a sink node s and connect it to all nodes having positive demand. The capacity of the edge is equal to d_i .



■

Assume that we have a lower bound on edge (B, E) of value $\ell_{BE} = 2$. This means that in a given solution, there should be at least 2 units flowing on the edge.

Answer 11 We use a similar transformation as used in min-cost flow problems. Reduce the capacity of edge (B, E) from 3 to 1. Increase demand at node B from 0 to 2. Decrease demand at node E from 0 to -2. This leads to the model:

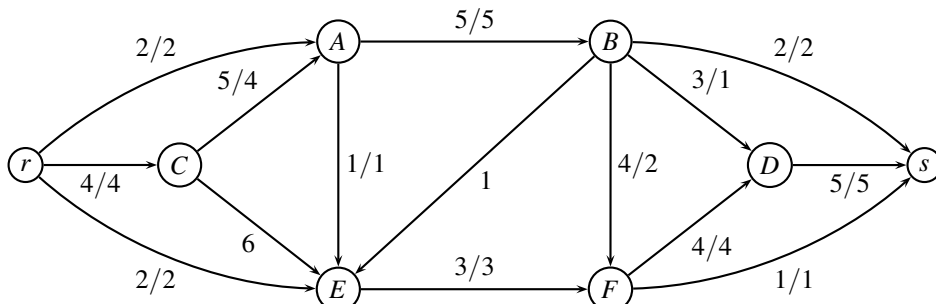


■

Answer 12 We use the augmenting path method, with the following paths:

- 4 items on $rCABFDs$
- 2 items on $rEFBs$
- 1 item on $rABDs$
- 1 item on $rAEFs$

This results in the following flow



■

Answer 13 The maximum flow can be found by summing flow from r , which is $z = 2 + 4 + 2 = 8$.

We find the minimum cut by marking all nodes that are reachable from r . This becomes $R = \{r\}$ and $\bar{R} = \{A, B, C, D, E, F, s\}$. The capacity of the cut is $c(R) = 2 + 4 + 2 = 8$. So, the solution is optimal.

Notice that for a circulation problem (if feasible), a minimum cut will be across all edges going out from r , or across all edges going into s . ■

Hungarian Algorithm

In the *Assignment problem* we are asked to assign a number of resources to an equal number of activities, so as to minimize total cost of allocation. Each resource should be assigned to exactly one activity, and each activity should have exactly one resource assigned.

The assignment problem is frequently solved with the *Hungarian Algorithm*: Given a cost matrix $C = \{c_{ij}\}$ saying how much it costs to assign resource i to activity j . The first step of the algorithm is to subtract the smallest number from each column, then subtract the smallest number from each row.

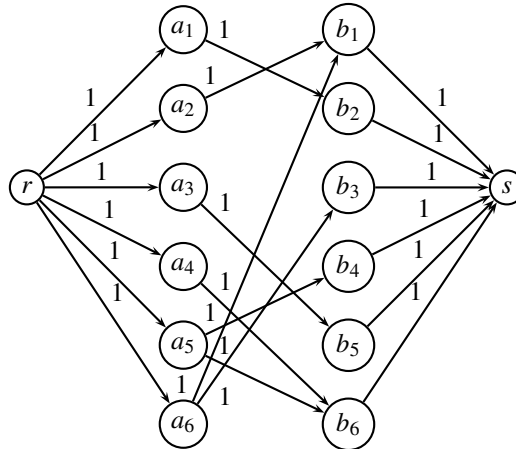
Next, we need to find the largest set of *independent* 0-values. Two 0-values are *independent* if they do not appear in the same row or column. For instances, in the first matrix below the bold-face 0-values are independent, but not in the second matrix

$$\begin{pmatrix} . & \mathbf{0} & . & . & . & . \\ \mathbf{0} & . & . & . & . & . \\ . & . & . & . & \mathbf{0} & . \\ . & . & . & . & . & \mathbf{0} \\ . & . & . & 0 & . & 0 \\ 0 & . & \mathbf{0} & . & . & . \end{pmatrix} \qquad \begin{pmatrix} . & \mathbf{0} & . & . & . & . \\ 0 & . & . & . & . & . \\ . & . & . & . & 0 & . \\ . & . & . & . & . & \mathbf{0} \\ . & . & . & 0 & . & \mathbf{0} \\ \mathbf{0} & . & \mathbf{0} & . & . & . \end{pmatrix} \quad (1)$$

Answer 14 We formulate it as a *maximum flow problem* with root r and sink s .

Introduce a node a_i for each row i in matrix C , and introduce a node b_j for each column j in matrix C . Connect the root r to a_i with edge capacity 1. Connect node b_j to the sink s with edge capacity 1.

For each zero value in the matrix $c_{ij} = 0$ connect node a_i and b_j using an edge with capacity 1.



If a maximum flow solution has flow on edge (a_i, b_j) then the corresponding zero-value in the matrix ($c_{ij} = 0$) should be selected. ■

Repositioning of empty containers

Consider the MEDITLINE network from the project assignment. In a given month, we have the following inventory of empty containers:

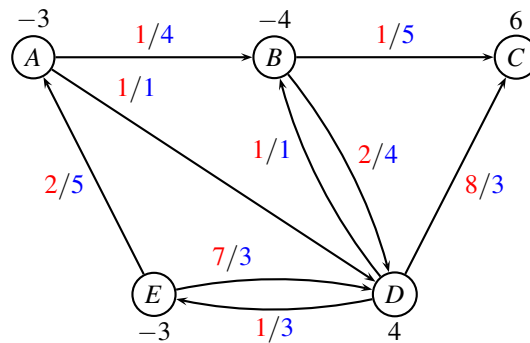
node	A	B	C	D	E
inventory	3	4	2	4	3

All empty containers are equal. We want to reposition the empty containers such that all containers are moved to C and D . Furthermore, the two nodes C and D should have the same amount of containers after repositioning.

Answer 15 We have 16 containers in total, so C and D should have 8 each after repositioning. This results in the following demands:

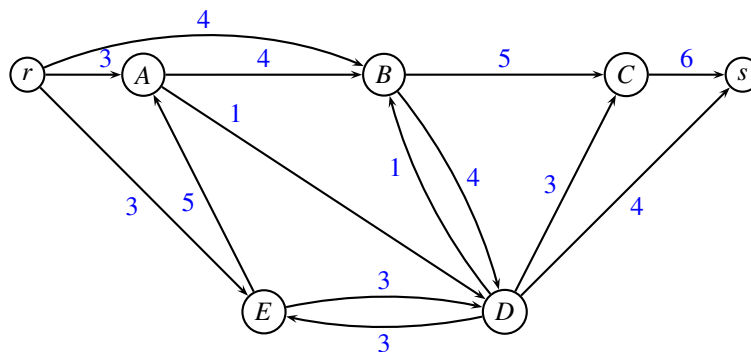
node	A	B	C	D	E
demand	-3	-4	6	4	-3

Adding demands to the graph gives the min-cost-flow problem:



Answer 16 We use the *augmenting cycle* algorithm to solve the min-cost flow problem.

To find an initial flow we solve a maximum flow problem. We first add a root r and a sink s . The root r is connect to all nodes with negative demand, with edge capacity equal to the demand. All nodes with positive demand are connected to the sink s , with edge capacity equal to the demand. In this way we get:

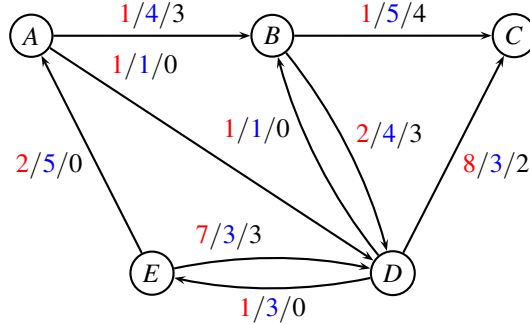


We solve the maximum flow problem with the *augmenting path* algorithm:

- 4 units $rBCs$
- 3 units $rABDs$

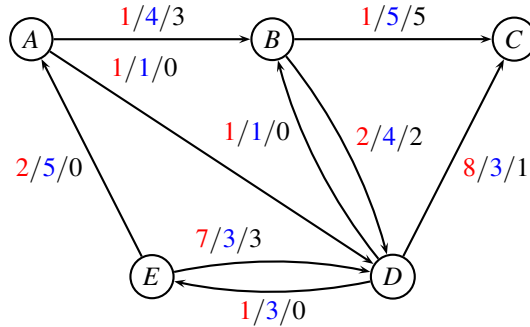
- 2 units $rEDCs$
- 1 units $rEDs$

Now, we have the flow:

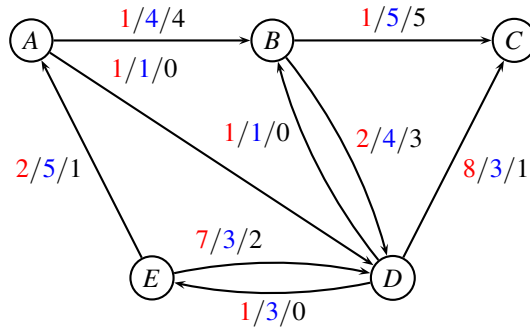


The cost of the flow is $z = 1 \cdot 3 + 1 \cdot 4 + 2 \cdot 3 + 8 \cdot 2 + 7 \cdot 3 = 50$.

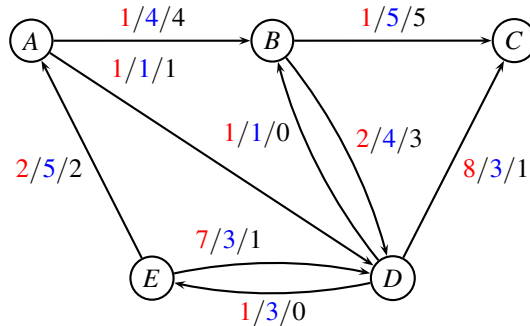
An augmenting cycle is BCD with cost $1 - 8 - 2 = -9$. The bottleneck is $\delta = \min\{1, 2, 3\} = 1$. We send 1 unit along the cycle getting:



An augmenting cycle is $ABDE$ with cost $1 + 2 - 7 + 2 = -2$. The bottleneck is $\delta = \min\{1, 2, 3, 5\} = 1$. We send 1 unit along the cycle getting:



An augmenting cycle is $ADEA$ with cost $1 - 7 + 2 = -4$. The bottleneck is $\delta = \min\{1, 2, 4\} = 1$. We send 1 unit along the cycle getting:



No more negative cycles can be found, so we terminate. The cost of the flow is $z = 1 \cdot 4 + 1 \cdot 5 + 2 \cdot 2 + 1 \cdot 1 + 2 \cdot 3 + 8 \cdot 1 + 7 \cdot 1 = 35$ ■

Checking:

```

minimize 1 xAB + 1 xAD + 1 xBC + 2 xBD + 1 xDB + 8 xDC + 1 xDE + 2 xEA + 7 xED
subject to
    xEA - xAB - xAD = -3
    xAB + xDB - xBC - xBD = -4
    xBC + xDC = 6
    xAD + xBD + xED - xDB - xDC - xDE = 4
    xDE - xEA - xED = -3

    xAB <= 4
    xAD <= 1
    xBD <= 4
    xBC <= 5
    xDB <= 1
    xDC <= 3
    xDE <= 3
    xEA <= 5
    xED <= 3
end

```

```

CPLEX> opt
Dual simplex - Optimal: Objective = 3.5000000000e+01
CPLEX> dis sol var -
Variable Name      Solution Value
xAB                4.000000
xAD                1.000000
xBC                5.000000
xBD                3.000000
xDC                1.000000
xEA                2.000000
xED                1.000000
All other variables in the range 1-9 are 0.

```

Flowing containers

We consider again the route net MEDITLINE from project assignment 2023, in which we have *new demands* as follows:

k	s_k	t_k	d_k
1	E	D	3
2	D	C	1
3	E	C	2

Commodity $k = 1$ is shipping three containers from E to D , commodity $k = 2$ is shipping one container from D to C , and commodity $k = 3$ is shipping two containers from E to C .

We are considering a *unsplittable multi-commodity flow problem* in which all the d_k containers must follow the same path.

Answer 17 We have the routes r outlined in the below table. The cost of sending d_k units along a route r is calculated as $d_k \sum_{(i,j) \in r} c_{ij}$.

number r	commodity k	route r	d_k	$\sum c_{ij}$	cost
1	1	$EABD$	3	5	15
2	1	ED	3	7	21
3	2	DBC	1	2	2
4	2	DC	1	8	8
5	2	$DEABC$	1	5	5
6	3	$EABC$	2	4	8
7	3	$EABDC$	2	13	26
8	3	EDC	2	15	30

■

We solve the LP-relaxed problem through *column generation*. Let y_{ij} denote the dual variable corresponding to the capacity constraint for edge (i, j) , and let $\bar{y}_k$ denote the dual variable corresponding to the demand constraint for commodity k .

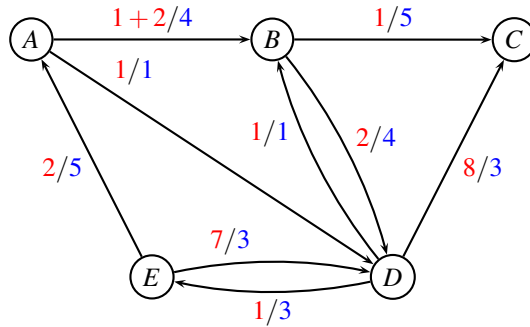
The dual variables for the above model are: $y_{AB} = -2$, while all other are $y_{ij} = 0$. Furthermore, $\bar{y}_1 = 21$, $\bar{y}_2 = 8$, $\bar{y}_3 = 30$.

Answer 18 The reduced cost of a route r is calculated as

$$\bar{c}_r^k = \sum_{(i,j) \in r} d_k c_{ij} - \sum_{(i,j) \in r} d_k y_{ij} - \bar{y}_k = \sum_{(i,j) \in r} d_k (c_{ij} - y_{ij}) - \bar{y}_k$$

In order to solve the pricing-problem as a shortest path problem we set the cost of edge (i, j) to $\tilde{c}_{ij} = d_k(c_{ij} - y_{ij})$. For commodity k we first remove all edges with capacity $u_{ij} < d_k$, as they cannot accommodate the demand. Next, we solve the shortest path problem using edge costs $\tilde{c}_{ij}$, with root s_k and destination t_k . Let z_k be the length of the shortest path for commodity k .

Since $y_{AB} = -2$ and remaining $y_{ij} = 0$, we get the graph:



Edges with capacity less than d_k are removed when solving the shortest path problem for commodity k . In this way we get:

$k = 1$: We remove edges $(AD), (DB)$. The shortest path is ED (route 2) with cost $z = 7$. Reduced cost is $d_1 \cdot z - \bar{y}_1 = 3 \cdot 7 - 21 = 0$.

$k = 2$: No edges are removed. The shortest path is DBC (route 3) with cost $z = 2$. Reduced cost is $d_2 \cdot z - \bar{y}_2 = 1 \cdot 2 - 8 = -6$.

$k = 3$: We remove edges $(AD), (DB)$. The shortest path is $EABC$ (route 6) with cost $z = 6$. Reduced cost is $d_3 \cdot z - \bar{y}_3 = 2 \cdot 6 - 30 = -18$.

■

Answer 19 The path with most negative reduced cost is added to the problem, which in our case is EABC (route 6). ■

Checking:

```

minimize
+15x1+21x2+2x3+8x4+11x5+8x6+26x7+30x8
subject to
UAB: +3x1          +1x5+2x6+2x7      <= 4
UAD:              <= 1
UBC:          +1x3      +1x5+2x6      <= 5
UBD: +3x1          +2x7      <= 4
UDB:          +1x3          <= 1
UDC:              +1x4      +2x7+2x8 <= 3
UDE:              +1x5          <= 3
UEA: +3x1          +1x5+2x6+2x7      <= 5
UED:      +3x2          +2x8 <= 3

K1: +x1+x2 = 1
K2: +x3+x4+x5 = 1
K3: +x6+x7+x8 = 1

\\ R1: x1 <= 0
\\ R2: x2 <= 0
R3: x3 <= 0
\\ R4: x4 <= 0
R5: x5 <= 0
R6: x6 <= 0
\\ R7: x7 <= 0
R8: x8 <= 0
end

```

```

CPLEX> opt
Dual simplex - Optimal: Objective = 5.1000000000e+01

CPLEX> dis sol var -
Variable Name      Solution Value
x1                  0.666667
x2                  0.333333
x4                  1.000000
x7                  1.000000
All other variables in the range 1-8 are 0.
CPLEX> dis sol dua -
Constraint Name      Dual Price
UAB                  -2.000000
K1                   21.000000
K2                    8.000000
K3                   30.000000
R3                   -6.000000
R6                  -18.000000
All other dual prices in the range 1-16 are 0.

```

_____end of solution_____